

ASISTENCIAQR

RESPUESTA TÉCNICA AL ANÁLISIS DE CHATGPT

Sistema Escolar de Asistencia por Código QR

Análisis basado en revisión directa del código fuente · Marzo 2026

1. CONTEXTO Y PROPÓSITO DE ESTE DOCUMENTO

Este documento responde punto a punto al análisis que ChatGPT realizó sobre los archivos **index.html** y **apoderado.html** del sistema AsistenciaQR. A diferencia de ese análisis —basado en lectura visual de los archivos—, este informe parte de una **inspección directa del código fuente**, incluyendo la lógica JavaScript de registro, deduplicación, caché, validación de horarios y manejo de autenticación.

El objetivo es separar lo que es un **riesgo real confirmado en el código** de lo que es una suposición incorrecta o un riesgo sobreestimado, para que el colegio tome decisiones informadas y concretas.

2. LO QUE CHATGPT ACERTÓ

Los siguientes puntos están **confirmados en el código fuente** y representan riesgos o limitaciones reales:

✓ Contraseña inicial = DNI del alumno	CONFIRMADO en index.html (línea ~3373) y apoderado.html (línea ~1462). El sistema crea la cuenta del apoderado con el DNI como contraseña inicial y además guarda un campo passBackup con la contraseña en texto dentro de Firestore. Esto es un riesgo de seguridad real.
✓ Toda la lógica está en el cliente (sin Cloud Functions)	CONFIRMADO. No existe ningún archivo de backend, Cloud Function ni servidor intermedio. Toda la validación (horario, deduplicación, tipo de registro) ocurre en el navegador. Esto significa que un usuario con conocimientos técnicos podría manipular el flujo.
✓ Nombre del colegio y eslogan fijados en el HTML	CONFIRMADO. Las variables COLEGIO_NOMBRE y COLEGIO_ESLOGAN se leen de Firestore/config, pero hay valores por defecto en el código fuente. Esto no afecta el funcionamiento, pero es una rigidez de despliegue.

✓ Colas en horas pico son el riesgo operativo real

CONFIRMADO. El flujo de escaneo requiere enfocar la cámara sobre el QR del alumno, lo que toma 1-3 segundos por persona. Con 1,300 alumnos en pocos minutos, el cuello de botella será físico/operativo, no necesariamente técnico.

3. LO QUE CHATGPT SE EQUIVOCÓ O SOBREESTIMÓ

Los siguientes puntos del análisis de ChatGPT son **incorrectos o parcialmente incorrectos** según el código real:

■ "No hay deduplicación fuerte en backend"

El sistema SÍ tiene deduplicación. El objeto **_hoyRegs** (índice en memoria) registra por alumno si ya marcó ingreso y/o salida hoy. Antes de cualquier escritura en Firestore, se consulta ese índice (líneas 4577-4585 de index.html). Si ya marcó ambos, muestra "Ya registró ingreso y salida hoy" y no escribe nada. Es deduplicación en cliente, no en servidor, pero funciona correctamente en uso normal con un solo operador por turno.

■ "El flujo es interactivo por alumno — confirmar ingreso"

El flujo actual NO requiere confirmación manual. La función **autoRegistrar()** determina automáticamente si corresponde INGRESO o SALIDA según la hora, y registra directamente en Firestore. No hay paso de "confirmar" que frene el flujo. Esto hace el proceso más rápido de lo que ChatGPT estimó.

■ "Tablas lentas y frontend pesado para 1,300 alumnos"

El sistema usa un **caché en memoria** (DB.getAlumnos()) que carga la lista de alumnos una sola vez y la reutiliza. Los reportes y filtros trabajan sobre datos ya cargados localmente. Firestore no es consultado en cada escaneo si los datos ya están en caché. Esto es un diseño deliberado y correcto para reducir lecturas de base de datos.

■ "El sistema podría congelarse bajo carga masiva"

Firebase en plan Spark (gratuito) soporta hasta 1 escritura/segundo por documento y 1 millón de escrituras/día. Con 1,300 registros de ingreso distribuidos en ~30 minutos (turno mañana), la tasa es de aproximadamente 0.7 escrituras/segundo, bien dentro del límite. Un congelamiento total es altamente improbable; la lentitud sería marginal y solo en picos extremos.

4. MATRIZ DE RIESGO — ESTADO REAL DEL SISTEMA

Evaluación de riesgos basada en el código fuente, con probabilidad e impacto para una operación de 1,300 alumnos:

RIESGO	PROBABILIDAD	IMPACTO	MEDIDA CORRECTIVA
Contraseña inicial = DNI (passBackup en Firestore)	ALTA	ALTO	Forzar cambio de contraseña en primer acceso (ya implementado). Eliminar campo passBackup.
Duplicados por acceso simultáneo desde dos dispositivos	MEDIA	MEDIO	Implementar transacción de Firestore o Cloud Function para validar en servidor.
Colas en hora de ingreso (problema operativo, no técnico)	ALTA	MEDIO	Múltiples puntos de escaneo o registro por aula/docente. Escalonar por turno.
Lentitud de interfaz en PCs/celulares de gama baja	MEDIA	BAJO	Probar en dispositivos reales del colegio antes del despliegue.
Manipulación del registro desde el navegador (sin backend)	BAJA	ALTO	Agregar Cloud Functions para validar tipo y hora en servidor. Implementar reglas Firestore.
Caída de Firestore / problemas de conectividad	MUY BAJA	ALTO	Modo offline con Service Worker y sincronización posterior.
Consumo de cuota gratuita de Firestore (Spark plan)	BAJA	BAJO	1,300 registros/día = ~0.13% del límite diario. Sin problema.

5. VEREDICTO TÉCNICO REAL

¿Puede funcionar para 1,300 alumnos?

Sí, puede funcionar — pero el análisis de ChatGPT mezcló riesgos reales con suposiciones incorrectas sobre el código. Aquí el resumen honesto:

- ✓ **Técnicamente sólido para la escala:** Firebase + caché local + deduplicación en memoria hacen que el backend soporte bien 1,300 alumnos si la operación está bien distribuida.
- ✓ **El flujo de registro es rápido:** No hay paso de confirmación; `autoRegistrar()` decide y guarda directamente, lo que permite escanear un alumno cada 2-3 segundos.
- **El riesgo operativo real es la concentración:** Si 600 alumnos intentan ingresar en 10 minutos por 1 solo punto de escaneo, el problema será la cola física, no la base de datos.
- **Seguridad de contraseñas necesita mejora:** El campo `passBackup` almacena la contraseña en Firestore. Debe eliminarse. El forzado de cambio de contraseña en primer acceso ya está implementado, lo cual es positivo.

- **Sin validación en servidor:** Toda la lógica corre en el navegador. Un usuario técnico podría manipularla. Para uso escolar normal esto es aceptable; para alta seguridad se necesitarían Cloud Functions.
- ✓ **Apto para producción con ajustes operativos menores:** No requiere rediseño técnico completo. Con múltiples puntos de escaneo y la corrección del passBackup, el sistema es viable para 1,300 alumnos.

6. ACCIONES CONCRETAS RECOMENDADAS (por prioridad)

PRIORIDAD ALTA

- Eliminar el campo **passBackup** de Firestore en todos los documentos de apoderados.
- Documentar y comunicar al personal que la contraseña inicial es el DNI y debe cambiarse.
- Instalar al menos 2-3 puntos de escaneo para el turno de mayor concurrencia.

PRIORIDAD MEDIA

- Agregar reglas de seguridad en Firestore (Security Rules) para limitar escrituras por rol.
- Probar el sistema con todo el personal antes del primer día de clases con alumnos.
- Escalonar el ingreso por grado o turno para evitar concentración en 1 sola franja.

PRIORIDAD BAJA / MEJORA FUTURA

- Implementar Cloud Function para validar registro en servidor (elimina riesgo de manipulación).
- Agregar modo offline con sincronización automática al recuperar conexión.
- Considerar QR dinámico (token temporal) si se requiere mayor seguridad del código QR.

Este análisis fue generado por **Claude Code (Anthropic)** mediante inspección directa del código fuente de AsistenciaQR en marzo de 2026. Las líneas de código referenciadas corresponden a los archivos **index.html** y **apoderado.html** del repositorio. github.com/moisesohts1007/asistencia-qr